Rockchip_Development_Guide_IRFPA_CN

文件标识：RK-YH-YF-634

发布版本：V0.1.3

日期：2025-08-25

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文旨在描述RKOOC（Rockchip On-Chip Offset Calibration）模块的作用，整体工作流程，及相关的API接口。主要给驱动工程师调试IRFPA提供帮助。

读者对象

本文档（本指南）主要适用于以下工程师：

驱动开发工程师

系统集成软件开发工程师

适用平台及系统

芯片名称	软件系统	支持情况
RV1126B	Linux Kernel-6.1	Y

修订记录

版本号	作者	修改日期	修改说明
v0.1.0	杨栋博	2025-08-25	初始版本

目录

Rockchip_Development_Guide_IRFPA_CN

1. 概述
    - 1.1 典型应用框图
    - 1.2 总体软件流程图
2. OOC硬件介绍
    - 2.1 OOC硬件接口
    - 2.2 工作模式
        - 2.2.1 完整模式
        - 2.2.2 数据模式
    - 2.3 RV1126B
        - 2.3.1 框图
3. IRFPA内核驱动介绍
    - 3.1 IRFPA相关内核驱动结构
    - 3.2 RKVICAP 驱动
    - 3.3 RKOOC 驱动
        - 3.3.1 驱动代码
        - 3.3.2 框架说明
        - 3.3.3 DTS 配置
        - 3.3.4 media结构
        - 3.3.5 驱动大致流程说明
        - 3.3.6 rkocc-subdev 命令和结构体说明
    - 3.4 探测器驱动
        - 3.4.1 DTS 配置
        - 3.4.2 media结构
        - 3.4.3 探测器驱动移植说明
4. IRFPA用户层APP介绍
    - 4.1 概述

4.2 irfpa_dv 参考例程介绍
4.2.1 VICAP模块介绍
4.2.2 OOC模块介绍
4.2.3 图像处理流程介绍
4.3 IRFPA-ISP 库介绍
4.3.1 各算法概述
4.3.2 API 描述

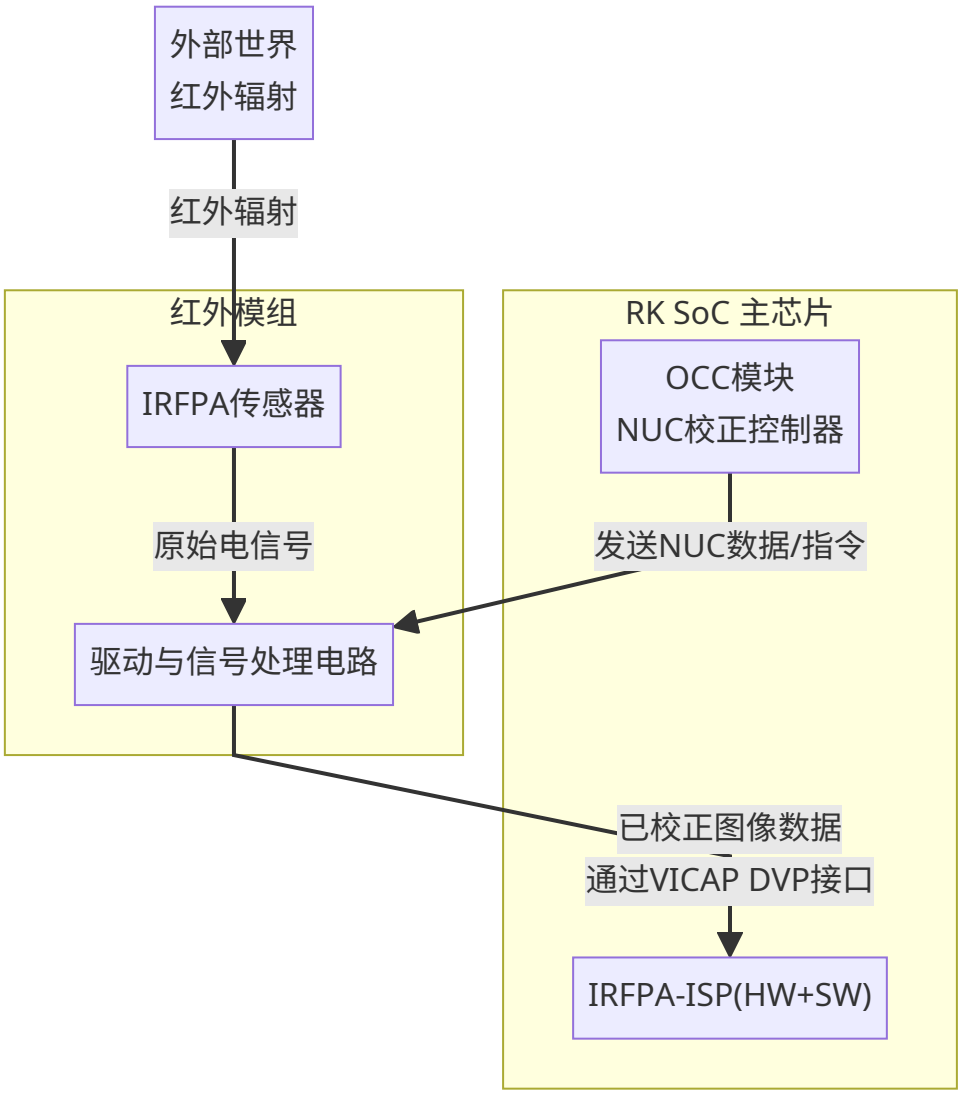
1. 概述

红外焦平面阵列（Infrared Focal Plane Array, IRFPA）是红外成像系统的核心传感器件，其功能是将入射的红外辐射转换为电信号，并直接输出由成千上万个探测元组成的二维图像数据。IRFPA集光电转换、信号积分与多路读取于一体，具有高灵敏度、高分辨率和快速响应等特点，是现代热成像相机、夜视仪、安防监控以及医疗诊断等设备的关键组成部分。

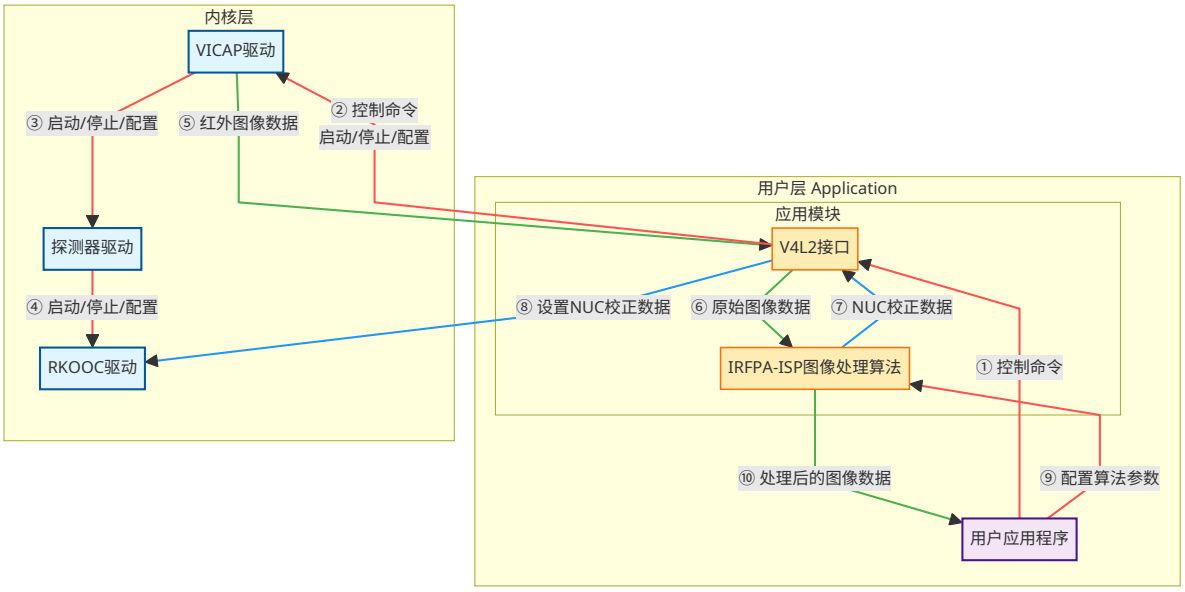
为实现高质量的红外成像，必须对IRFPA固有的非均匀性进行校正。为此，系统采用非均匀性校正（Non-Uniformity Correction, NUC）技术。在RK平台解决方案中，具体的数据交互流程如下：主控系统芯片（System on Chip, SoC）通过Soc中集成的OOC(On-chip Offset Calibration)模块，将计算得到的NUC校正系数发送至红外模组。模组接收并应用这些数据，对其输出的原始图像数据进行实时校正。

校正后的红外图像数据流，则通过并行的数字视频端口（Digital Video Port, DVP）接口，从红外模组传输至SoC。SoC接收此图像数据流后，可使用软件或专用硬件进行后续的图像增强、降噪、伪彩渲染以及最终的编码与输出等一系列图像处理流程，从而完成整个红外图像的采集、校正与显示。

1.1 典型应用框图



1.2 总体软件流程图



数据流类型说明

数据流类型	颜色示例	说明
控制命令流		用于传输系统控制指令和配置信息
图像数据流		用于传输原始和处理后的图像数据
NUC校正数据流		用于传输非均匀性校正数据

2. OOC硬件介绍

OOC (On-chip Offset Calibration) 模块是SoC内部专用于连接并驱动非制冷红外探测器（Uncooled IRFPA）的关键硬件单元。其主要功能是生成并输出红外探测器实时成像所需的信号与指令。该模块通过特定的控制总线与红外模组通信，将校正数据或时序控制信号发送至模组内的驱动电路，以补偿IRFPA像元固有的非均匀性缺陷，确保输出原始图像数据的质量。

2.1 OOC硬件接口

信号名称	方向	类型	描述
irfpa_mclk	OCC → 模组	时钟	主时钟信号
irfpa_fsync	OCC → 模组	同步	帧同步信号（可选）。
irfpa_sda[0:3]	OCC → 模组	数据	并行数据线。

2.2 工作模式

OOC 模块支持两种不同的工作模式，用于向红外模组传输NUC数据和配置信息。这两种模式通过内部寄存器进行选择和配置。

2.2.1 完整模式

该模式指的是OOC模块输出探测器寄存器配置和NUC信号,寄存器配置在帧同步期间输出。
(详细说明待实现)

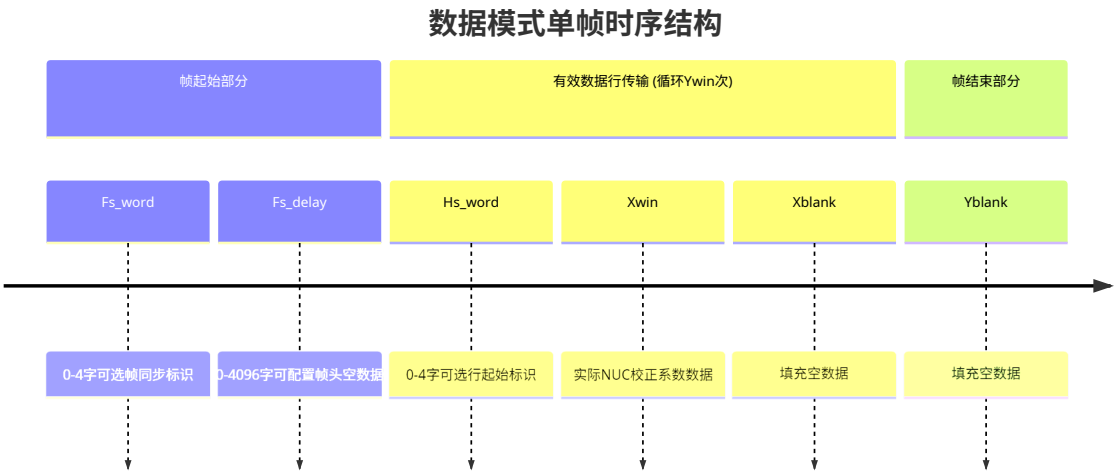
2.2.2 数据模式

该模式指的是OOC模块仅输出NUC信号。探测器配置由其它接口完成(例如I2C)。
在数据模式下，irfpa_fsync 信号不被使用,数据传输完全通过数据包结构进行同步。

NUC信号参数定义表

参数符号	参数名称	单位	描述
Fmclk	主时钟频率	Hz	OCC模块输出给红外模组的时钟频率
fps	帧率	frames/s	系统要求的图像刷新率
Xwin	有效像素列数	pixels	探测器水平方向有效像元数
Xblank	水平空白像素	pixels	每行传输结束后的水平消隐时间
Ywin	有效像素行数	lines	探测器垂直方向有效像元数
Yblank	垂直空白行数	lines	每帧传输结束后的垂直消隐时间
Fs_word	帧同步字	pixels	帧头同步标识
Hs_word	行同步字	pixels	行头同步标识
Fs_delay	帧同步间隔	pixels	帧同步字到第一行行同步字的空白间隔

单帧数据传输时序图



详细时序说明

1. 每帧数据只能从帧同步部分开始发送
2. 每个数据需要两个clk分别传输高3bit和低3bit
3. Fs_delay,Xblank,Yblank期间只能发送固定值(可配置)
4. OCC输出时钟频率必须与帧格式匹配

MCLK 频率计算说明

基于上述参数，主时钟频率的计算公式为：

```
# 水平总像素数计算
Xtotal_size = Hs_word_size + Xwin + Xblank

# 垂直总行数计算
Ytotal_size = Ywin + Yblank
```

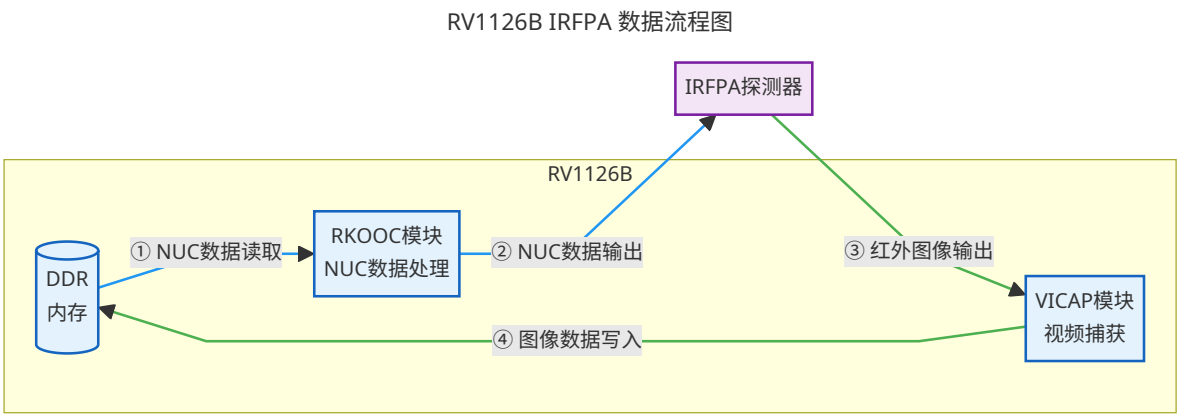
```
# 帧同步开销计算
Fsync_size = Fs_word_size + Fs_delay

# 主时钟频率计算
# 单帧总像素数 = (单行像素数 × 单帧行数) + 帧同步开销
# 每秒总像素数 = 单帧总像素数 × 帧率
# 主时钟频率 = 每秒总像素数 × 2
Fmclk = 2 × fps × [(Xtotal_size) × (Ytotal_size) + Fsync_size]
```

2.3 RV1126B

RV1126B平台支持一路OOC数据输出

2.3.1 框图



性能及约束概述

参数项	描述	取值范围
Xtotal_size	行头+有效数据+行尾填充	<4096
Ytotal_size	有效行+空行数	<4096行
Fmclk	MCLK最大频率	<150Mhz

3. IRFPA内核驱动介绍

3.1 IRFPA相关内核驱动结构

IRFPA相关内核驱动主要有RKOOC驱动, VICAP驱动,探测器驱动

```
# RK平台IRFPA项目目录结构

## 内核层代码
kernel/
├── arch/arm/boot/dts/              # arm32 DTS配置文件
├── arch/arm64/boot/dts/rockchip/   # arm64 DTS配置文件
│   └── rv1126b-evb-cam-irfpa.dtsi  # rv1126b evb1 IRFPA相关DTS配置文件
├── drivers/media/
│   ├── platform/rockchip/cif/      # VICAP驱动目录
│   ├── platform/rockchip/occ/      # RKOOC驱动目录
└── i2c/
    └── gst412c.c          # 参考探测器驱动
```

3.2 RKVICAP 驱动

对于RV1126B平台而言,VICAP和OOC是独立的两个图像处理IP。OCC模块输出探测器所需的NUC和配置信号,VICAP模块负责接收探测器输出数据)。VICAP驱动说明参见RKVICAP驱动。

3.3 RKOOC 驱动

RKOOC（Rockchip On-Chip Offset Calibration）驱动是基于标准的 V4L2/Media 框架实现的硬件控制模块，主要负责硬件资源配置、中断服务处理、数据缓冲区轮转管理等核心功能。

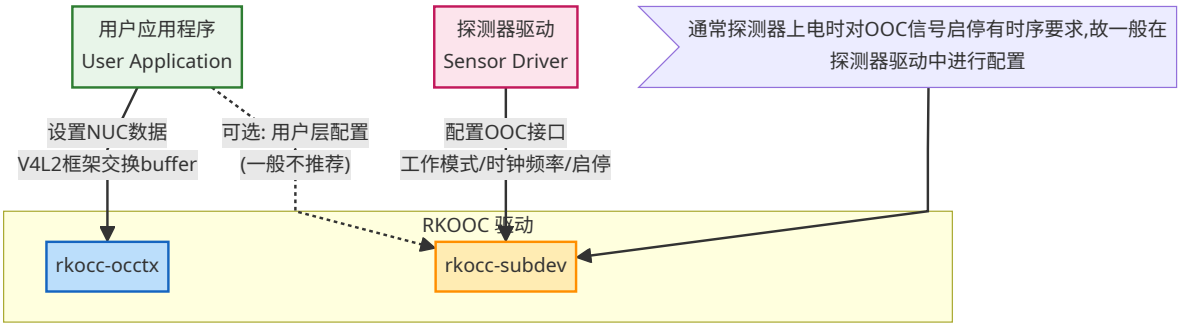
3.3.1 驱动代码

```
drivers/media/platform/rockchip/occ
    common.c            # 通用辅助函数
    dev.c               # RKOOC platfrom,v4l2, meida设备注册入口
    hw.c                # 硬件寄存器配置函数
    Makefile            # 内核驱动编译脚本
    occ.c               # rkocc-tx 子设备定义
    regs.h              # 硬件寄存器定义
    rkocc-externel.h    # subdev call相关结构体定义
    subdev.c            # rkocc-subdevice 子设备定义
```

3.3.2 框架说明

RKOOC 驱动根据功能分为两个部分:

- rkocc-occtx 负责数据缓冲区轮转管理。通过 V4L2 框架提供的标准机制，将包含校正数据的Buffer从用户空间安全、高效地传递至内核空间。
- rkocc-subdev 一般由探测器驱动配置,根据探测器需求配置ooc接口、工作模式、时钟频率、启动停止等。鉴于红外探测器在上电初始化及正常工作过程中对 OOC 信号的电平时序有严格要求，此类配置通常集成在探测器驱动中，由上电序列统一管理，以确保硬件工作的可靠性。



3.3.3 DTS 配置

以RV1126B_EVB1举例:

```
rkocc0: rkocc-tx@0x22160000 {
    reg = <0x22160000 0x100>;
    status = "okay";
    compatible = "rockchip,rkocc-tx";

    pinctrl-names = "default";
    pinctrl-0 = <&ir_fpa_pins>;

    interrupts = <GIC_SPI 133 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "occ-irq";

    clocks = <&cru DCLK_OOC_SRC>, <&cru ACLK_OOC>, <&cru HCLK_OOC>;
    clock-names = "pmclk", "aclk", "hclk";

    assigned-clocks = <&cru DCLK_OOC_SRC>;
    assigned-clock-parents = <&cru PLL_GPLL>;
    assigned-clock-rates = <13500000>;
};
```

DCLK_OOC_SRC 时钟配置说明

`DCLK_OOC_SRC` 是 OOC 模块输出给红外探测器的主时钟（MCLK）信号源，,探测器不同配置会对应不同的频率,需要在此处正确配置。

在RV1126B平台,此时钟只能由GPLL或CPLL分频得到

```
// RV1126B 平台时钟源选择约束
#define GPLL_FREQ     1188000000  // GPLL 频率: 1.188GHz
#define CPLL_FREQ     1000000000  // CPLL 频率: 1.0GHz

// 目标频率: 13.5MHz
#define TARGET_FREQ    13500000

// 分频计算
#define GPLL_DIVIDER   (GPLL_FREQ / TARGET_FREQ)   // 88 分频 → 13.5MHz
#define CPLL_DIVIDER   (CPLL_FREQ / TARGET_FREQ)   // 74 分频 → 13.514MHz
```

此处为了适配13500000频率,选择GPLL进行分频。

不同SOC可能对应不同的PLL频率,需要查阅SOC文档或SDK选择合适的时钟源。

3.3.4 media结构

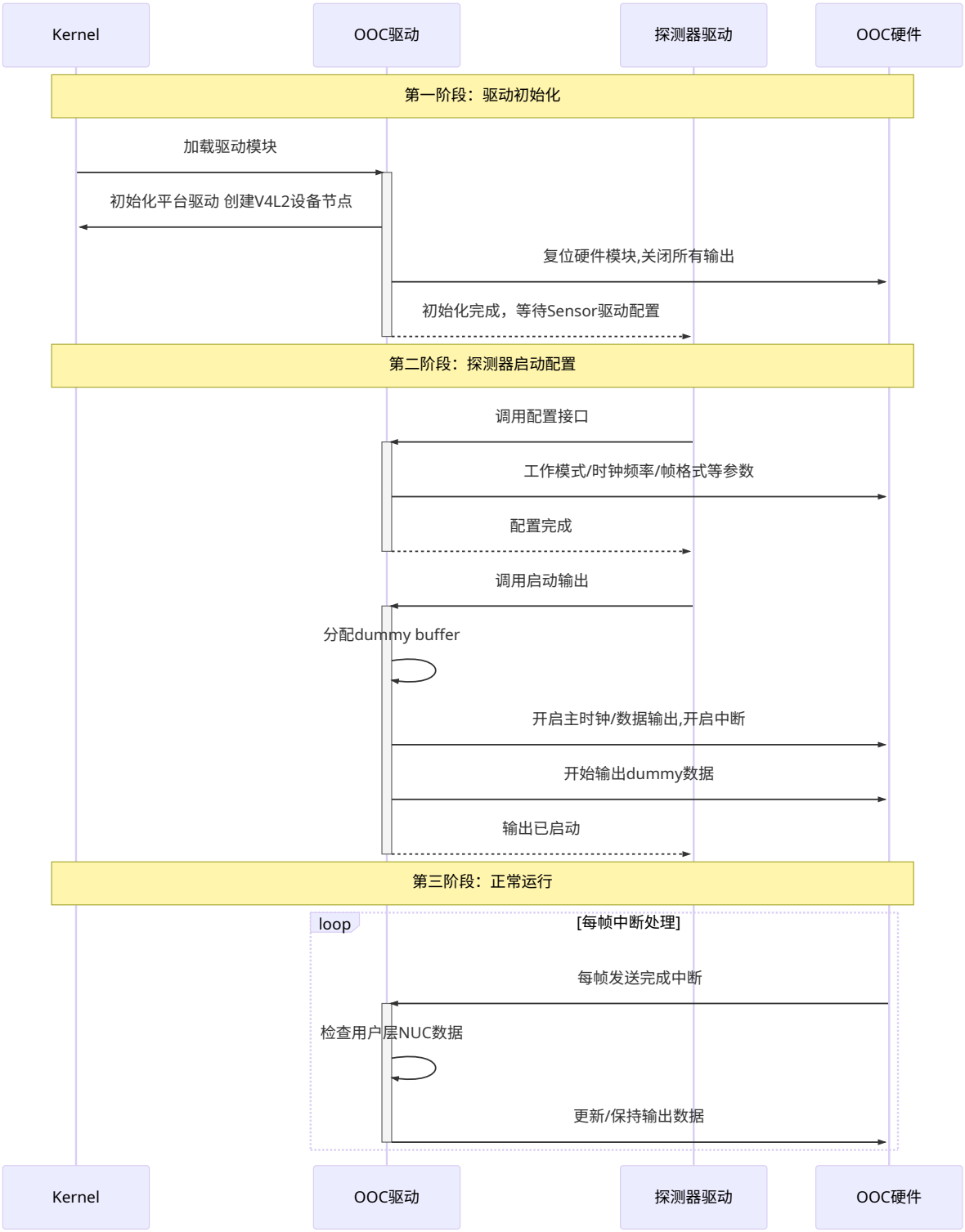
DTS配置后且config中配置了RKOOC之后系统启动后可以用media-ctl 命令看到RKOCC的media结构

```
Device topology
- entity 1: rkocc-occtx (1 pad, 0 link)
            type Node subtype V4L flags 0
            device node name /dev/video1
        pad0: Source

- entity 5: rkocc-subdev (1 pad, 0 link)
            type V4L2 subdev subtype Unknown flags 0
            device node name /dev/v4l-subdev0
        pad0: Source
```

名称	类型	描述
rkocc-occtx	Video Output	用户层设置NUC数据内容
rkocc-subdev	Sub Device	探测器驱动配置OOC帧格式,开关流等

3.3.5 驱动大致流程说明



3.3.6 rkocc-subdev 命令和结构体说明

IOCTL CMD	描述	参数结构体
RKOOC_CMD_CONFIG_FULLMODE	完整模式配置命	struct rkocc_config_fullmode
RKOOC_CMD_CONFIG_DATAMODE	数据模式配置命令	struct rkocc_config_datamode

```
// 完整模式配置结构体
struct rkocc_config_full {
    u32 fmclk;  // 主时钟频率
    // TODO: 其他完整模式特有参数
};

// 数据模式配置结构体
struct rkocc_config_datamode {
    u32 fmclk;          // 主时钟频率

    u16 xwin_size;      // 有效像素列数
    u16 ywin_size;      // 有效像素行数
    u16 xblank_size;    // 水平消隐像素数
    u16 yblank_size;    // 垂直消隐行数

    u16 hs_word_size;   // 行同步字长度
    u32 hs_word;        // 行同步字内容
    u16 fs_word_size;   // 帧同步字长度
    u32 fs_word;        // 帧同步字内容

    u16 fs_delay_size;  // 帧同步延迟

    u8 blank_key;       // 消隐期间数据标识
};
```

3.4 探测器驱动

红外探测器驱动需要添加rkocc-subdev控制OOC接口,此外与可见光CIS驱动相同。此处只说明与OOC驱动交互的部分,其余可参考Rockchip_Driver_Guide_VI,本文档不再赘述。

3.4.1 DTS 配置

以RV1126B平台,GST412C为例,为了与OOC驱动交互,需要在DTS中添加 rkocc = <&rkocc0>;
其余与普通DVP接口sensor的配置方式一致。

```
gst412c: gst412c@42 {
    status = "okay";
    reg = <0x42>;

    // 指向rkocc驱动
    rkocc = <&rkocc0>;
```

3.4.2 media结构

DTS配置后,系统启动后会进入probe流程,如果成功则会出现探测器media节点。可以用media-ctl 命令看到RKOCC的media结构

```
- entity 45: m00_f_gst412c 3-0042 (1 pad, 1 link)
             type V4L2 subdev subtype Sensor flags 0
             device node name /dev/v4l-subdev1
        pad0: Source
                [fmt:unknown/400x308@10000/250000 field:none
                 crop.bounds:(0,4)/400x300]
                -> "stream_cif_dvp_id0":0 [ENABLED]
```

3.4.3 探测器驱动移植说明

1. 获取ooc-subdev句柄

以GST412C为例, 首先需要在注册驱动时获取rkocc驱动句柄,如果DTS中配置生效此处就可以正确找到。成功获取后保存在sensor结构体中后续使用。

```c
// drivers/media/i2c/gst412c.c
struct gst412c {
    // ...
    struct v4l2_subdev *rkocc_sd;  // OOC驱动句柄
};

// 在probe函数中获取句柄
static int gst412c_probe(struct i2c_client *client,
                         const struct i2c_device_id *id)
{
    struct gst412c *gst412c;
    struct device_node *rkocc_node = NULL;
    struct platform_device *rkocc_pdev = NULL;

    // 从DTS获取rkocc节点
    rkocc_node = of_parse_phandle(node, "rkocc", 0);
    if (rkocc_node) {
        rkocc_pdev = of_find_device_by_node(rkocc_node);
        of_node_put(rkocc_node);
        if (rkocc_pdev) {
            gst412c->rkocc_sd = platform_get_drvdata(rkocc_pdev);
        }
    }
    // ...
}
```

OCC接口配置

启动OCC接口输出前需要先选择工作模式,并配置OCC帧格式,通过ioctl完成

```c
struct rkocc_config_datamode config = {
    .fmclk = 13500000,     // 13.5MHz主时钟

    .xwin_size = 640,      // 640列有效像素
    .ywin_size = 512,      // 512行有效像素
    .xblank_size = 32,     // 水平消隐32像素
    .yblank_size = 16,     // 垂直消隐16行
```

```
    .hs_word_size = 2,      // 2字节行同步字
    .hs_word = 0xAA55,      // 行同步字内容
    .fs_word_size = 4,      // 4字节帧同步字
    .fs_word = 0x5A5AA5A5, // 帧同步字内容

    .fs_delay_size = 8,     // 8像素帧同步延迟
    .blank_key = 0xFF,      // 消隐数据标识
};
// 通过ioctl配置OOC驱动,设置为 data mode
v4l2_subdev_call(occ_sd, core, ioctl,
    RKOOC_CMD_CONFIG_DATAMODE, &config);
```

启动/停止主时钟输出

如果探测器上电时需要在接收OOC信号前先打开主时钟,可以通过调用subdev call s_crystal_freq 来配置时钟。其中第一个参数GST412C_PMCLK_FREQ是主时钟频率,第二个参数为1代表开启主时钟输出,为0代表停止主时钟输出。

```
// 设置时钟频率并启动主时钟输出
v4l2_subdev_call(gst412c->rkocc_sd, video, s_crystal_freq, GST412C_PMCLK_FREQ,
1);

// 关闭主时钟输出
v4l2_subdev_call(gst412c->rkocc_sd, video, s_crystal_freq, GST412C_PMCLK_FREQ,
0);
```

启动/停止OOC输出

探测器需要帧同步信号时,可以通过调用subdev_call s_stream 控制OCC输出。OOC启动时会先分配一个dummy buffer,buffer内为可配置固定值,第一帧固定输出dummy buffer。每帧OOC数据输出完成后会进入中断,此时若用户层设置了新的buffer则会复制新buffer的内容输出,否则保持输出当前buffer。

```
// 启动OOC数据流输出
v4l2_subdev_call(gst412c->rkocc_sd, video, s_stream, 1);
// 停止OOC数据流输出
v4l2_subdev_call(gst412c->rkocc_sd, video, s_stream, 0);
```

修改NUC校准数值

探测器驱动只能配置第一帧默认输出的固定OOC值,此后无法再修改NUC校准数据,只能通过用户层操作rkocc-tx节点修改。具体参见用户层APP说明。

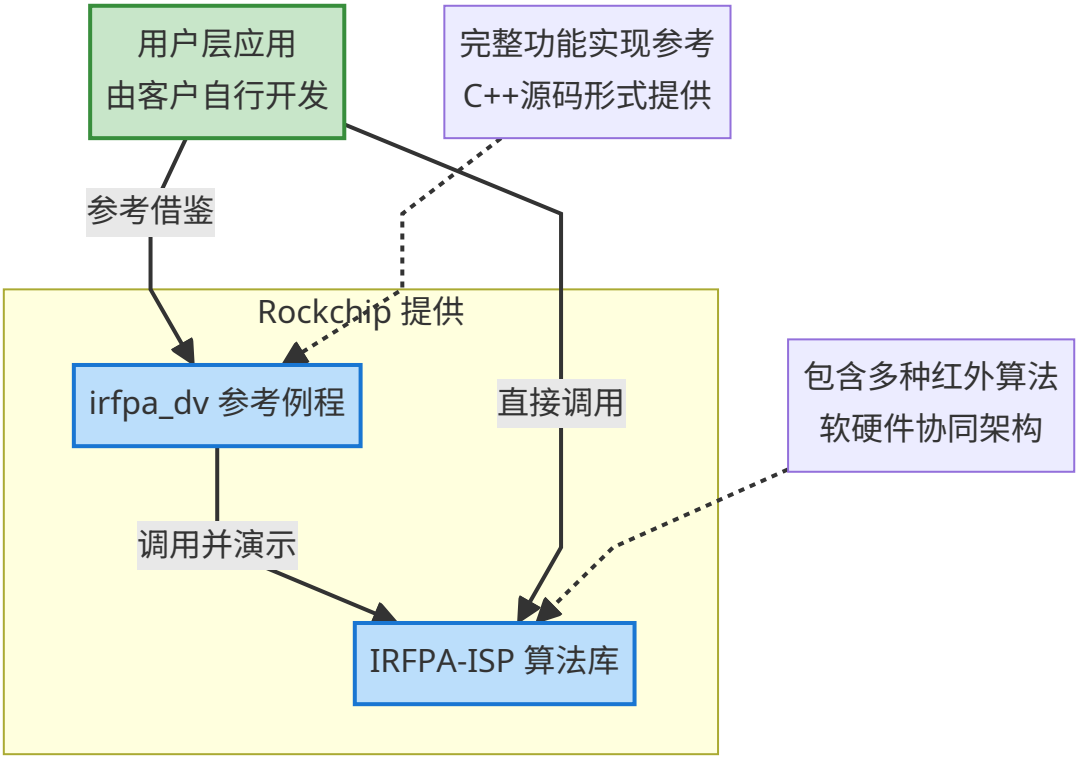
4. IRFPA用户层APP介绍

4.1 概述

用户层应用由客户自行开发,RK提供`IRFPA-ISP`算法库与 `irfpa_dv` 参考例程。

`IRFPA-ISP` 是 Rockchip 自主开发的红外图像处理算法库，集成了时域降噪（BTNR)、细节增强（Sharp)、去坏点（DPCC)、去条纹（Destripe)、非均匀校正（NUC）及伪彩映射（Pseudo）等红外图像处理算法。该算法库采用软硬件协同架构，关键算法通过硬件加速实现，显著提升处理效率。

`irfpa_dv` 是基于 C++ 开发的综合性参考例程，以源码形式提供完整的功能实现参考。该例程通过 V4L2 接口实现红外图像采集、探测器参数配置及 NUC 校正数据设置；演示了 IRFPA-ISP 算法库的调用流程与集成方法；采用 LVGL 库构建用户界面，支持实时图像显示与交互控制；基于 TOML 格式配置文件实现算法参数管理；并集成 TCP 网络传输功能，支持将红外图像流及温度数据远程传输至其它设备。



4.2 irfpa_dv 参考例程介绍

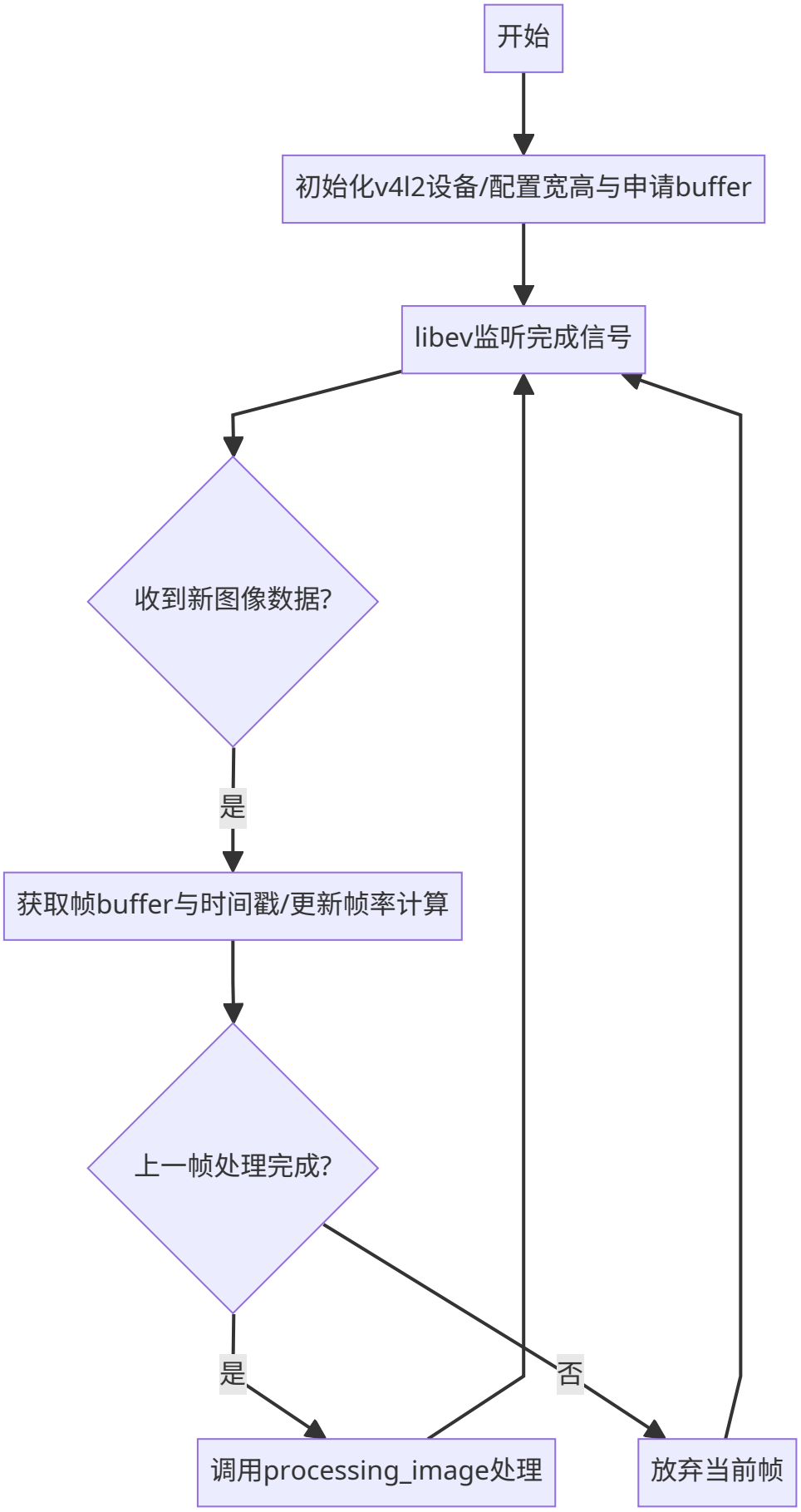
4.2.1 VICAP模块介绍

VICAP模块负责红外图像的采集与预处理，其主要流程如下：

1. **设备发现与初始化**
   - 遍历系统media节点，定位IRFPA对应的video-input设备
   - 初始化video-in设备，配置图像宽度高度参数
   - 申请DMA缓冲区用于图像数据存储
2. **红外数据流监听**

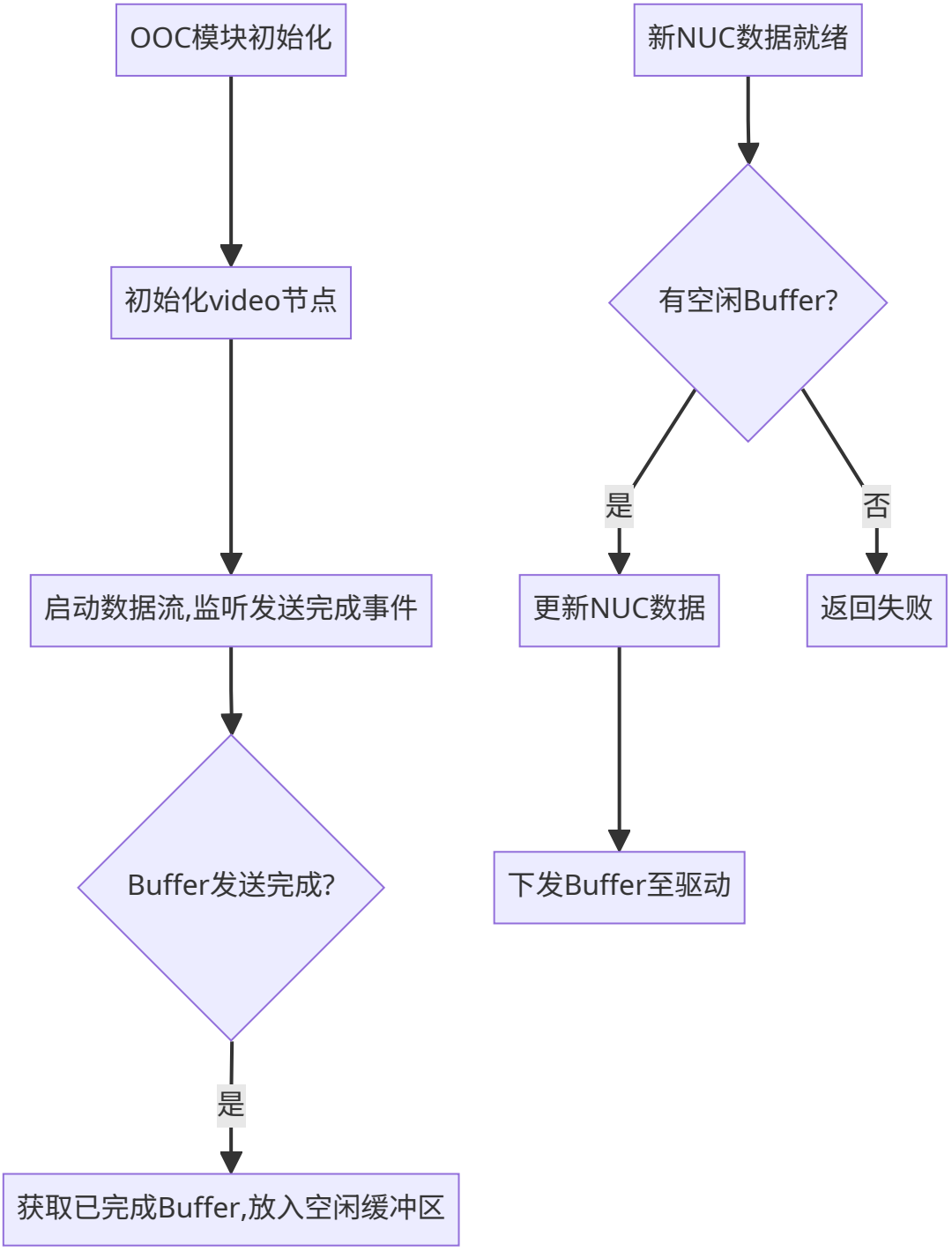
- 调用stream_on启动红外图像流采集
- 使用libev事件循环监听数据接收完成信号

3. **数据流控制**
    - 获取当前帧缓冲区数据并记录时间戳,计算并更新实时帧率统计
    - 检查上一帧处理状态，确保处理资源可用性
    - 调用processing_image函数进行图像处理

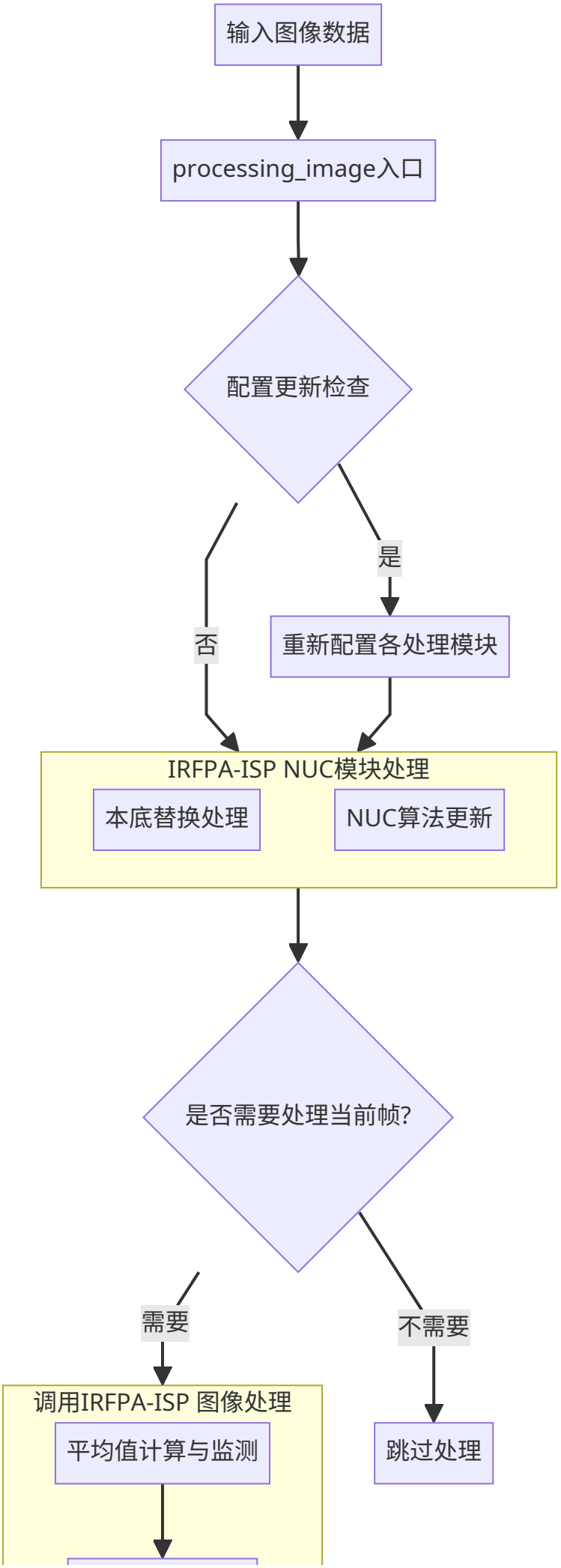

开始
初始化v4l2设备/配置宽高与申请buffer
libev监听完成信号
收到新图像数据?
是
获取帧buffer与时间戳/更新帧率计算
上一帧处理完成?
是
否
调用processing_image处理
放弃当前帧

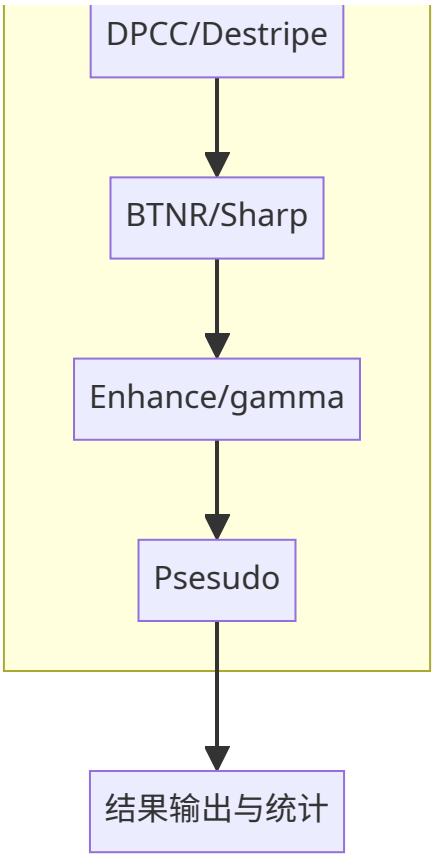
4.2.2 OOC模块介绍

1. **初始化阶段**
   设备初始化：配置OOC设备video节点参数
   资源分配：申请DMA缓冲区用于NUC数据传输
   流启动：调用start_stream启动数据输出流
2. **发送完成事件处理**
   事件监听：监控buffer发送完成事件
   缓冲区管理：将已完成传输的buffer回收到空闲队列
3. **NUC数据更新处理**
   NUC数据更新：接收图像处理流程生成的新校正数据
   资源检查：验证空闲缓冲区可用性
   数据下发：将填充NUC数据的buffer提交给驱动传输



4.2.3 图像处理流程介绍


输入图像数据
processing_image入口
配置更新检查
是
否
重新配置各处理模块
IRFPA-ISP NUC模块处理
本底替换处理
NUC算法更新
是否需要处理当前帧?
需要
不需要
调用IRFPA-ISP 图像处理
平均值计算与监测
跳过处理



4.3 IRFPA-ISP 库介绍

4.3.1 各算法概述

算法标识	算法名称	功能描述	实现方式	算法耗时
BTNR	时域降噪	基于多帧统计的时序噪声抑制	ISP	~2ms
DPCC	去坏点	检测并校正传感器固有坏点和热像素	软件实现	~ 2ms
Destripe	去条纹	消除因读出电路差异导致的条纹噪声	软件实现	~ 10ms
NUC	非均匀校正	实时补偿像元响应非均匀性	软件实现	~1ms
Sharp	细节增强	增强图像边缘和细节信息，改善视觉效果	ISP	~ 2ms
Enhance	对比度增强	优化图像灰度分布，提高对比度	混合实现	~ 10ms
Pseudo	伪彩映射	将灰度图像转换为伪彩色图像，增强视觉分辨力	软件实现	~ 2ms

4.3.2 API 描述